

Module 16: Files and IO

Two types do most of the work. `Path` is a **name**, and every method on it is pure string manipulation that touches no disk. `Files` is what actually reads, writes and asks questions of the filesystem.

Keeping those separate in your head prevents a surprising number of bugs, starting with the security one.

What You'll Learn

- `Path` as pure name manipulation, and the traversal bug it prevents
- The one-line `Files` methods for reading and writing
- Why `Files.lines` needs closing and `readString` does not
- What `java.io.File` gets wrong, with the failure shown side by side
- The exception family that names the file and the reason

Coming From Another Language

	Python	Java
Path object	<code>pathlib.Path</code>	<code>java.nio.file.Path</code>
Join	<code>/</code> operator	<code>.resolve(...)</code>
Read all	<code>path.read_text()</code>	<code>Files.readString(path)</code>
Read lines lazily	<code>with open(...) as f</code>	<code>Files.lines(path)</code> , must close
Missing file	<code>FileNotFoundException</code>	<code>NoSuchFileException</code>
Default encoding	UTF-8	UTF-8 since Java 18

`pathlib` and `java.nio.file` are close relatives, so most of this will feel familiar. The closing requirement on `Files.lines` is the one to watch.

The Lesson

Path is a name, not a file

From `PathBasics.java`. `resolve` appends, and gets the separator right on every platform:

```
Path.of("/var/data").resolve("reports/q3.csv")    // /var/data/reports/q3.csv
```

An absolute argument replaces the base entirely:

```
Path.of("/var/data").resolve("/etc/passwd")    // /etc/passwd
```

That behaviour is how unvalidated user input becomes a path traversal bug. The defence is to normalise and then check where you landed:

```
Path attempted = uploads.resolve(userSupplied).normalize();
attempted.startsWith(uploads)    // false for ../../etc/passwd
```

Use `Path.startsWith`, not `String.startsWith`. It compares name elements rather than characters, so it cannot be fooled:

```
String.startsWith would have said: true    // "/srv/uploads-evil/x"
Path.startsWith says:                  false
```

`normalize()` is pure and does **not** follow symlinks, so removing `b/..` can change which file you reach if `b` is a link. `toRealPath()` is the one that touches the disk, resolves links, and throws if the file is absent.

`Path.equals` is lexical. `Files.isSameFile` asks the filesystem.

Reading and writing

From `ReadingAndWriting.java`:

```
Files.writeString(path, "content");           // create or
        truncate
Files.writeString(path, "more\n", StandardOpenOption.APPEND); // append
String whole = Files.readString(path);
List<String> lines = Files.readAllLines(path);
```

Since Java 18 the default charset is UTF-8 everywhere. Before that the same code read different bytes on different machines, which produced a long tail of encoding bugs.

Useful queries: `exists`, `isRegularFile`, `isDirectory`, `size`, `isReadable`.

A note on `exists`. It answers a question about the past. The file can vanish before you act on the answer. Prefer attempting the operation and handling the exception over checking first. That race is called TOCTOU and it is a real source of bugs.

`Files.createDirectories` creates every missing parent. `copy`, `move` and `deleteIfExists` do what they say, and the copy options control overwriting.

Streaming, and the closing rule

`readString` and `readAllLines` load the **entire file**. Fine for a config file, fatal for a log.

From `StreamingLargeFiles.java`:

```
try (var lines = Files.lines(log)) {
    long errors = lines.filter(l -> l.startsWith("ERROR")).count();
}
```

`Files.lines`, `Files.list`, `Files.walk` and `Files.find` all hold an open file handle. The stream is the only thing that can close it, so try-with-resources is not optional. This is the most commonly leaked resource in Java file code, and on a busy server it means running out of descriptors.

Memory stays at one line regardless of file size, and the pipeline can short-circuit, so finding the first match reads only as far as it needs to.

One wrinkle when writing from inside a stream:

```
lines.forEach(line -> {
    try { out.write(line); } catch (IOException e) { throw new UncheckedIOException(e); }
});
```

A lambda cannot throw a checked exception. That is Module 10's complaint about checked exceptions and lambdas, met in the wild, and wrapping is the usual fix.

Why not java.io.File

From `WhyNotLegacyFile.java`, the same failure twice:

```
File.delete() returned: false
why did it fail? no permission? not there? locked?
the API does not say, and cannot be made to.

Files.delete() threw NoSuchFileException
naming the file: /var/.../does-not-exist.txt
```

That is the central complaint. Every `File` method that can fail returns `false`, so error handling becomes guesswork.

Three more: `File` cannot handle symbolic links, cannot read file attributes in one call, and `listFiles()` returns an **array**, so a directory with a million entries becomes a million-element array.

`path.toFile()` and `file.toPath()` convert freely. **Convert at the boundary with old APIs, and use `Path` everywhere inside your own code.**

The exception family

`MissingFile.java` throws `NoSuchFileException`, which carries the path as **structured data**:

```
catch (NoSuchFileException e) {  
    log.warn("missing config at {}", e.getFile());  
}
```

Exception	Meaning
<code>NoSuchFileException</code>	the path does not exist
<code>FileAlreadyExistsException</code>	it does, and you asked to create it
<code>AccessDeniedException</code>	permissions
<code>DirectoryNotEmptyException</code>	delete on a non-empty directory
<code>NotDirectoryException</code>	you treated a file as a directory

All are `IOException` subclasses. Catch the specific one when you can react differently, and `IOException` when you cannot.

Run It

```
java modules/16-files-and-io/examples/PathBasics.java  
java modules/16-files-and-io/examples/ReadingAndWriting.java  
java modules/16-files-and-io/examples/StreamingLargeFiles.java  
java modules/16-files-and-io/examples/WhyNotLegacyFile.java  
  
# Fails on purpose.  
java modules/16-files-and-io/examples/MissingFile.java  
  
./scripts/verify-examples.sh modules/16-files-and-io
```

Every example works in a temporary directory and deletes it before exiting, so running them leaves nothing behind.

Common Mistakes

Not closing `Files.lines` or `Files.walk`. A leaked file descriptor per call.

`readAllLines` on a large file. Use `Files.lines`.

Building paths with string concatenation. Use `resolve`, and normalise anything derived from user input.

`String.startsWith` for a containment check on paths. `/srv/uploads-evil` passes.
`Path.startsWith` does not.

Checking `exists` before every operation. The answer is stale the moment you have it.

Catching `Exception` around file work. The specific subclasses tell you whether to retry, create the file, or give up.

New code using `java.io.File`. Silent booleans instead of reasons.

Key Takeaways

- `Path` is a name; `Files` touches the disk. Path methods are pure.
- `resolve` with an absolute argument replaces the base, which is a traversal bug waiting to happen. Normalise and check with `Path.startsWith`.
- `readString` loads everything; `Files.lines` does not and must be closed.
- Every directory-walking method returns a closeable stream.
- Default charset has been UTF-8 since Java 18, so encoding no longer varies by machine.
- `Files` throws exceptions that name the file and the reason. `File` returns `false`.

Homework

[homework/README.md](#)

Build a log analyser that streams, then write a path validator that resists traversal. Reference solution in [solutions/16-files-and-io/](#).